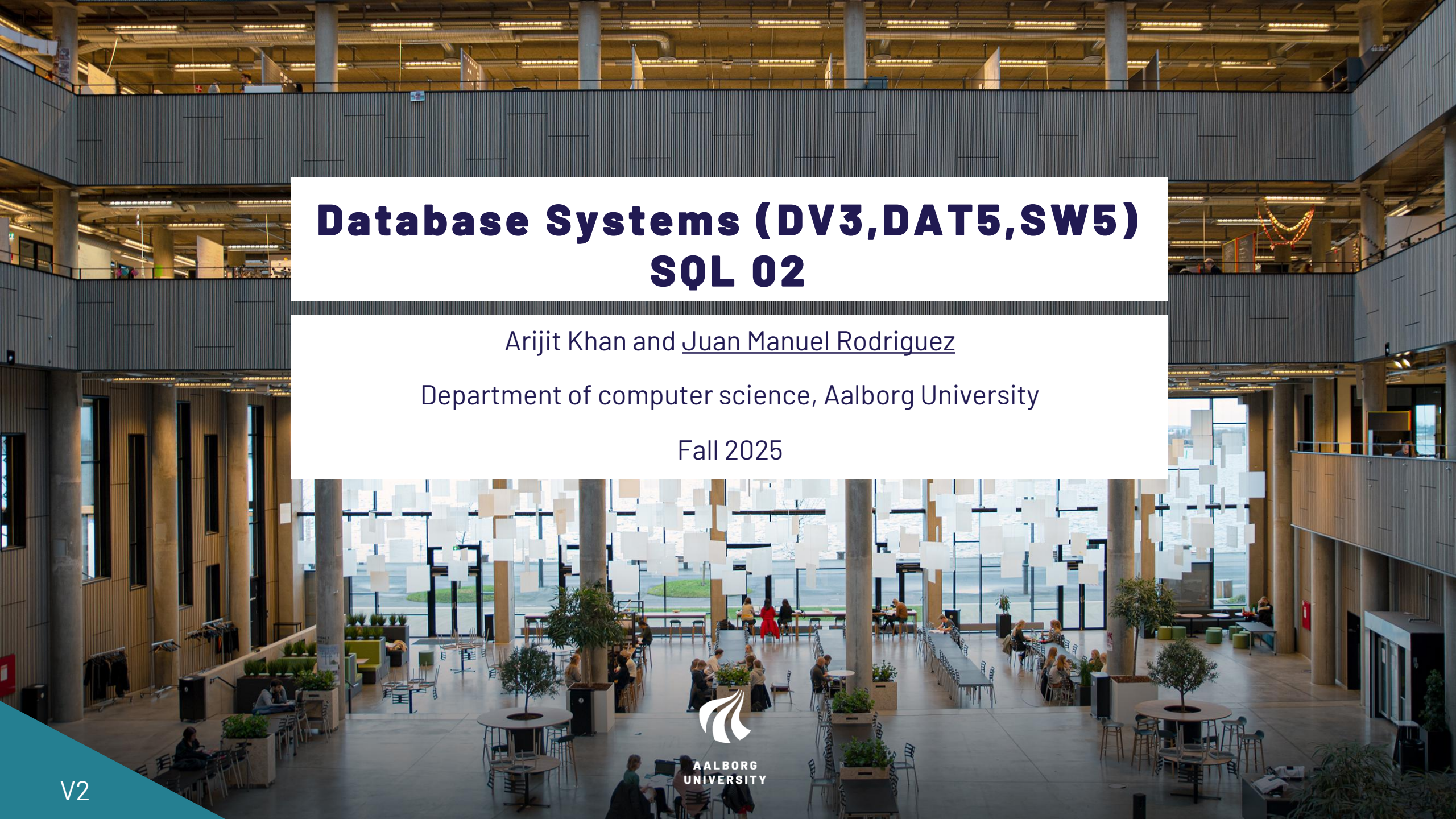
Database Systems (DV3,DAT5,SW5)
SQL 02

Arijit Khan and Juan Manuel Rodriguez

Department of computer science, Aalborg University

Fall 2025



AALBORG UNIVERSITY

V2

STOP ME!

If you have questions,

if you spot an error,

if something is different in the book,

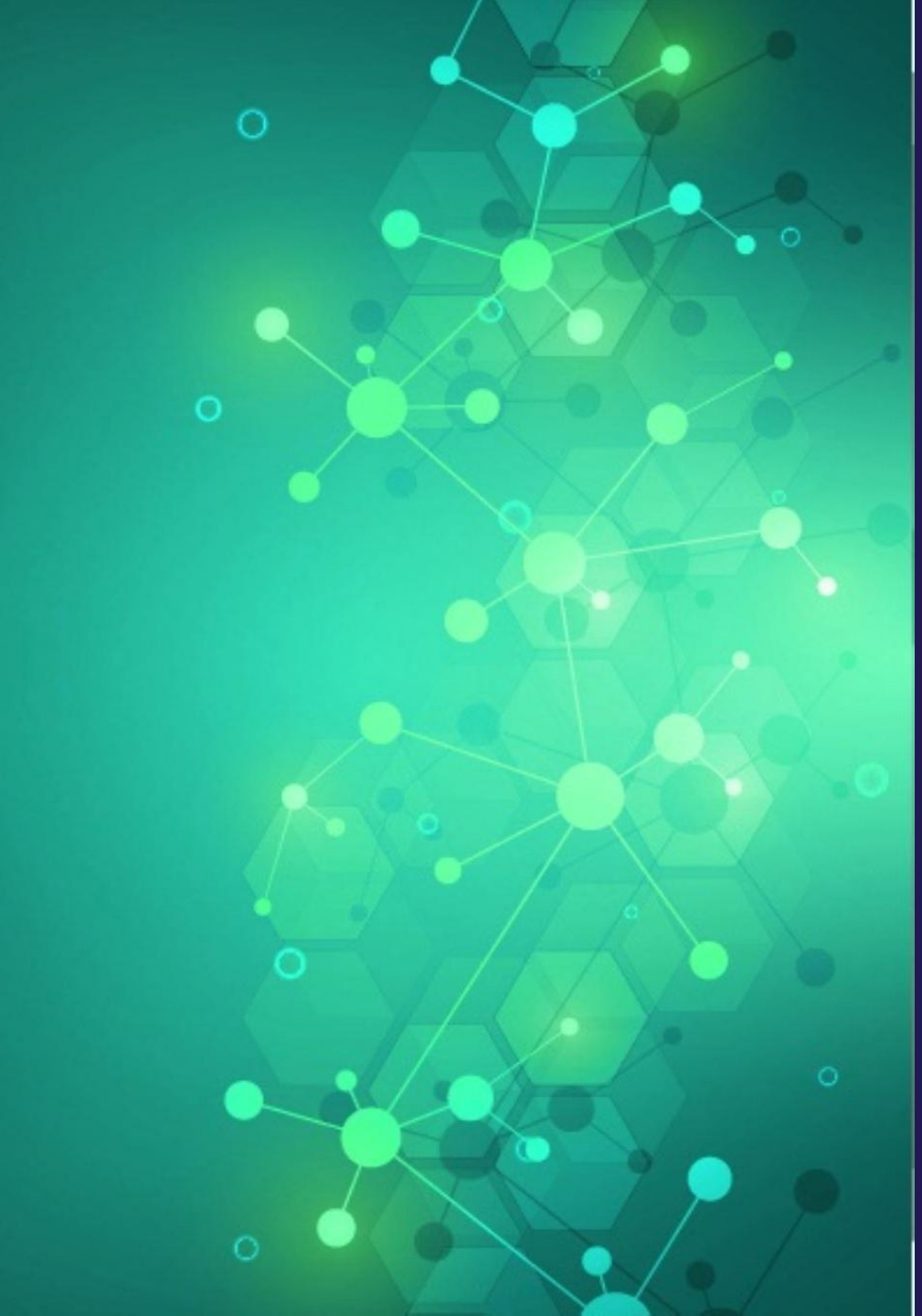
if you are unsure about something,

if you realize you are not following,

if something is on fire,

if I start talking too fast…

https://bit.ly/aau_dbs24



Thanks for the slides

- Gabriela Montoya
- Katja Hose
- Matteo Lissandrini

Outline last lecture

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples /Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

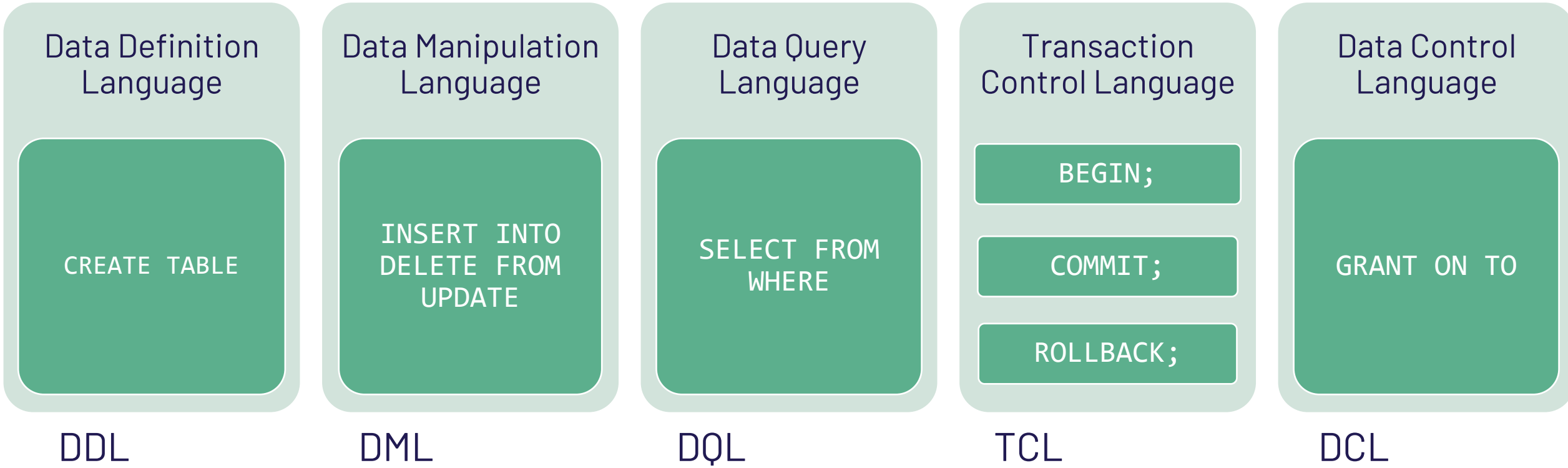
Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

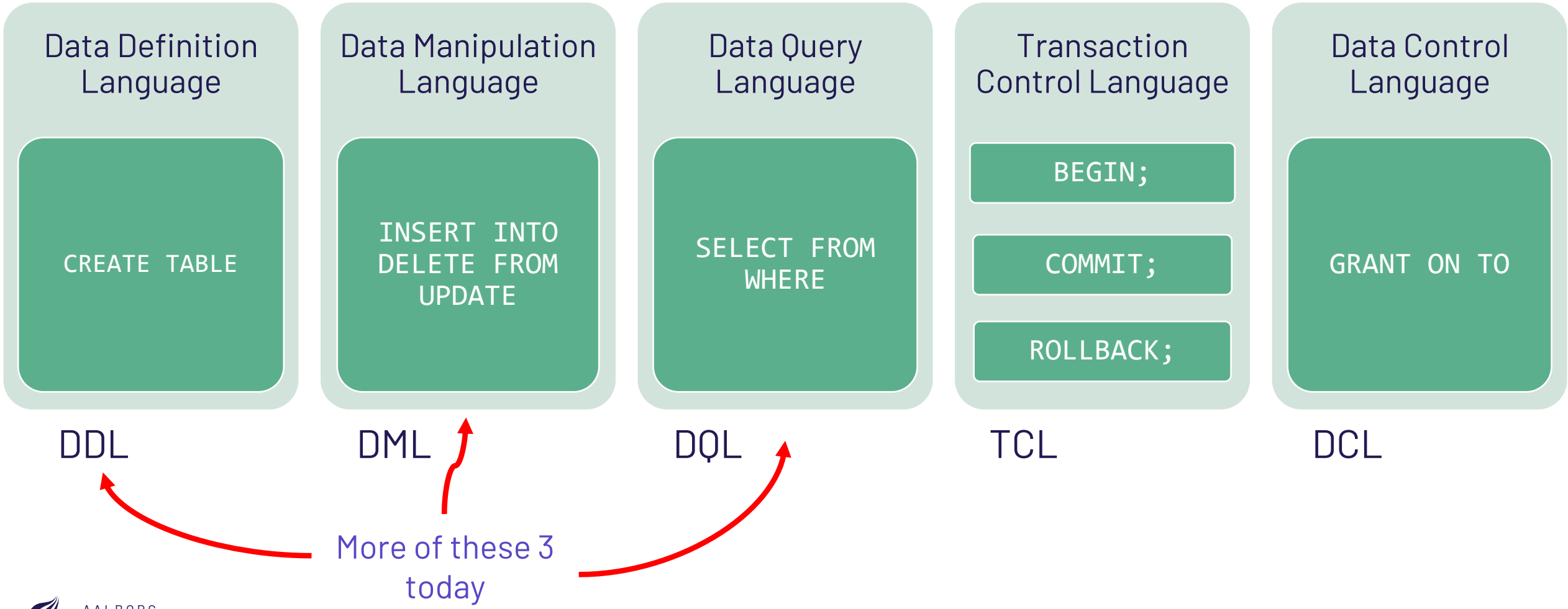
★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

SQL – Not only SELECT queries



SQL – Not only SELECT queries



Outline this lecture

Data Manipulation Language [part 2] ★

- Insert (+ select)
- Update (+where)
- Referential Integrity

Data Definition Language [part 2]

- CREATE (as query / with data) ★
- Sequence numbers
- Cascade / set NULL / set default ★

Data Query Language [part 2] ★

- LIMIT
- Nested queries / Uncorrelated subqueries
- Conditions IN / NOT IN / EXISTS / ALL / ANY
- INNER / OUTER / LEFT JOIN
- NULLs
- WITH statement

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

DML: INSERT

```
CREATE TABLE takes(
  studid integer REFERENCES student(studid),
  courseid integer REFERENCES course(courseid)
);
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8

course

courseid	title	ects	taughtBy
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

```
INSERT INTO takes
    SELECT studid, courseid
    FROM student, course
    WHERE title = 'Basics';
```

What is this doing now?

takes

studid	courseid
24002	5001
25403	5001
26120	5001
26830	5001

You can insert in a table the result of a query

Referential Integrity

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2137	Kant	C4

```
CREATE TABLE course (
    courseid integer PRIMARY KEY,
    title varchar(30) NOT NULL,
    ects integer,
    taughtby integer REFERENCES professor(id)
);
```

```
INSERT INTO course
    VALUES (9100, ‘Theory of Everything’, 2, 2125);
```

```
INSERT INTO course
    VALUES (5100, 'Spying for Dummies', 4, 1007);
```

```
INSERT INTO course
    VALUES (5100, 'Spying for Dummies', 4);
```

DML: UPDATE (I)

```
CREATE TABLE student (
    studid integer UNIQUE NOT NULL,
    name varchar(30) NOT NULL,
    semester integer
);
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

```
UPDATE student
SET semester=semester+1
WHERE semester < 18;
```

also updates cannot violate constraints

student

studid	name	semester
24002	Xenokrates	**18**
25403	Jonas	**13**
26120	Fichte	**11**
26830	Aristoxenos	**9**
27550	Schopenhauer	**7**
28106	Carnap	**4**
29120	Theophrastos	**3**
29555	Feuerbach	**3**

DML: UPDATE (II)

```
CREATE TABLE student (
    studid integer UNIQUE NOT NULL,
    name varchar(30) NOT NULL,
    semester integer
);
```

How can we update only 2 students?

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Incorrect statement

```
UPDATE student
SET semester=4
WHERE studid = 27550
      AND studid = 28106;
```

```
UPDATE student
SET semester=4
WHERE studid = 27550
      OR studid = 28106;
```

Correct statement

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	**4**
28106	Carnap	**4**
29120	Theophrastos	2
29555	Feuerbach	2

DML: Delete

How can we delete students?

```
CREATE TABLE student (
    studid integer UNIQUE NOT NULL,
    name varchar(30) NOT NULL,
    semester integer
);
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

```
DELETE FROM student
WHERE semester > 10;
```

student

studid	name	semester
~~24002~~	~~Xenokrates~~	~~18~~
~~25403~~	~~Jonas~~	~~12~~
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

DML: Delete (II)

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8

course

courseid	title	ects	taughtBy
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

```
CREATE TABLE takes(
  studid integer REFERENCES student(studid),
  courseid integer REFERENCES course(courseid)
);
```

✘ `DELETE FROM course WHERE coursed=5001;`

✓ `DELETE FROM course WHERE coursed=5043;`

✘ `DELETE FROM course;`

What is this doing now?

takes

studid	courseid
24002	5001
25403	5001
26120	5001
26830	5001

DELETING & FOREIGN KEYS

- TRUNCATE ***cannot be used if there is another table with a foreign key referencing the table*** we want to delete…
- The TRUNCATE TABLE statement is logically (though not physically) equivalent to the **DELETE FROM** statement (without a WHERE clause).
- TRUNCATE is in DDL while DELETE is in DML!! ← this matters!

REFERENTIAL INTEGRITY!
Foreign keys must always reference existing tuples or be NULL

COMMANDS in the DDL cannot CHECK whether the REFERENTIAL INTEGRITY is satisfied
ONLY statements in the DML can

DELETE FROM can be used* even if there is another table with a foreign key referencing the table, but the results depends on some additional constrains – see next!

*conditions apply!

Outline this lecture

Data Manipulation Language [part 2] ★

- Insert (+ select)
- Update (+where)
- Referential Integrity

Data Definition Language [part 2]

- CREATE (as query / with data) ★
- Sequence numbers
- Cascade / set NULL / set default ★

Data Query Language [part 2] ★

- LIMIT
- Nested queries / Uncorrelated subqueries ★
- Conditions IN / NOT IN / EXISTS / ALL / ANY
- INNER / OUTER / LEFT JOIN
- NULLs
- WITH statement

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

Outline this lecture

Data Manipulation Language [part 2] ★

- Insert (+ select)
- Update (+where)
- Referential Integrity

Data Definition Language [part 2]

- CREATE (as query / with data) ★
- Sequence numbers
- Cascade / set NULL / set default ★

Data Query Language [part 2] ★

- LIMIT
- Nested queries / Uncorrelated subqueries ★
- Conditions IN / NOT IN / EXISTS / ALL / ANY
- INNER / OUTER / LEFT JOIN
- NULLs
- WITH statement

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

SEQUENCE Number Generators

Sequence number generators automatically create continuous identifiers.

Can be used for automatic ***surrogate keys*** generation

```
CREATE SEQUENCE professorseq START 2100;

CREATE TABLE professor(
    id integer PRIMARY KEY DEFAULT NEXT VALUE FOR professorseq,
    name varchar(10) NOT NULL,
    rank char(2) DEFAULT 'C1'
);
```

```
In PostgreSQL:
DEFAULT nextval('professorseq')
```

Referential Integrity

```
CREATE TABLE takes(
  studid integer references student(studid),
  courseid integer references course(courseid)
);
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8

takes

studid	courseid
24002	5001
25403	5001

course

courseid	title	ects	taughtBy
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

If something happens to 5001 **what should happen to the table takes?**

Updates & Integrity Constraints

- Dynamic integrity constraints need to be fulfilled by each change of a database
- Possible responses to changes of referenced data
  - Rejection of updates (**default** behavior)
  - Propagation of updates (**CASCADE**)
  - Set references to “unknown” (**SET NULL**)
  - Set to a default value (**SET DEFAULT**)

Example of Referential Integrity

```
CREATE TABLE course (
    courseid integer PRIMARY KEY,
    title varchar(30) NOT NULL,
    ects integer,
    taughtby integer
       REFERENCES professor(id)
);
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

Assume we run one of the following:

```
UPDATE professor
   SET id = 2121
   WHERE id = 2125;
```

```
DELETE FROM professor
    WHERE id = 2125;
```

What to do now?

Option 1: REJECT the update

```
CREATE TABLE course (
    courseid integer PRIMARY KEY,
    title varchar(30) NOT NULL,
    ects integer,
    taughtby integer REFERENCES professor(id)
);
```

```
UPDATE professor
  SET id = 2121
  WHERE id = 2125;
```

```
DELETE FROM professor
  WHERE id = 2125;
```

Result of the operation:
DB remains unchanged!

Option 2: CASCADING updates

```
CREATE TABLE course (
    courseid integer PRIMARY KEY,
    title varchar(30) NOT NULL,
    ects integer,
    taughtby integer REFERENCES professor(id)
    ON UPDATE CASCADE ON DELETE CASCADE
);
```

```
UPDATE professor
  SET id = 2121
  WHERE id = 2125;
```

```
DELETE FROM professor
  WHERE id = 2125;
```

Result of the operations: some “updates” happen!

Option 2: Example of CASCADING updates

```
...  ON UPDATE CASCADE ...
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

```
UPDATE professor
   SET id = 2121
   WHERE id = 2125;
```

Result of the UPDATE operation:
BOTH Keys in professor and course are updated

professor

id	name	rank
2121	Socrates	C4
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	**2121**
5043	Theory of Cognition	3	2126

Option 2: Example of CASCADING deletes

```
... ON DELETE CASCADE ...
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

```
DELETE FROM professor
  WHERE id = 2125;
```

Result of the DELETE operation:
Tuples in professor and course get deleted

professor

id	name	rank
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5043	Theory of Cognition	3	2126

Use ON DELETE CASCADE carefully!

Option 3: update and SET NULL

```
CREATE TABLE course (
    courseid integer PRIMARY KEY,
    title varchar(30) NOT NULL,
    ects integer,
    taughtby integer REFERENCES professor(id)
    ON UPDATE SET NULL ON DELETE SET NULL
);
```

```
UPDATE professor
  SET id = 2121
  WHERE id = 2125;
```

```
DELETE FROM professor
   WHERE id = 2125;
```

Result of the operations: some “NULLs” appear!

Option 3: Example of SET NULL updates

... ON UPDATE **SET NULL** ...

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

```
UPDATE professor
   SET id = 2121
   WHERE id = 2125;
```

Result of the UPDATE operation:
empid set to 2121,
taughtby set to NULL

professor

id	name	rank
2121	Socrates	C4
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	⊥
5043	Theory of Cognition	3	2126

Option 3: Example of SET NULL deletes

```
... ON DELETE SET NULL ...
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

```
DELETE FROM professor
  WHERE id = 2125;
```

Result of the DELETE operation:
Tuple in professor deleted,
taughtby set to NULL

professor

id	name	rank
2126	Russel	C4
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	⊥
5043	Theory of Cognition	3	2126

Option 4: update and SET DEFAULT

```
CREATE TABLE course (
    courseid integer PRIMARY KEY,
    title varchar(30) NOT NULL,
    ects integer,
    taughtby integer DEFAULT 1 REFERENCES professor(id)
    ON UPDATE SET DEFAULT ON DELETE SET DEFAULT
);
```

```
UPDATE professor
  SET id = 2121
  WHERE id = 2125;
```

```
DELETE FROM professor
   WHERE id = 2125;
```

Result of the operations:
a specific key appears, only if valid!

Option 4: Example of SET NULL updates

```
... DEFAULT 1 ON UPDATE SET DEFAULT ...
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2137	Kant	C4
1	N.A.	XX

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

```
UPDATE professor
   SET id = 2121
   WHERE id = 2125;
```

Result of the UPDATE operation:
empid set to 2121,
taughtby set to 1

professor

id	name	rank
2121	Socrates	C4
2126	Russel	C4
2137	Kant	C4
1	N.A.	XX

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	**1**
5043	Theory of Cognition	3	2126

Option 4: Example of SET NULL updates

```
... DEFAULT 1 ON DELETE SET DEFAULT ...
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2137	Kant	C4
1	N.A.	XX

```
DELETE FROM professor
  WHERE id = 2125;
```

professor

id	name	rank
2126	Russel	C4
2137	Kant	C4
1	N.A.	XX

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

Result of the DELETE operation:
tuple in professor is deleted,
taughtby set to 1

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	**1**
5043	Theory of Cognition	3	2126

Outline this lecture

Data Manipulation Language [part 2] ★

- Insert (+ select)
- Update (+where)
- Referential Integrity

Data Definition Language [part 2]

- CREATE (as query / with data) ★
- Sequence numbers
- Cascade / set NULL / set default ★

Data Query Language [part 2] ★

- LIMIT
- Nested queries / Uncorrelated subqueries ★
- Conditions IN / NOT IN / EXISTS / ALL / ANY
- INNER / OUTER / LEFT JOIN
- NULLs
- WITH statement

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

Outline this lecture

Data Manipulation Language [part 2] ★

- Insert (+ select)
- Update (+where)
- Referential Integrity

Data Definition Language [part 2]

- CREATE (as query / with data) ★
- Sequence numbers
- Cascade / set NULL / set default ★

Data Query Language [part 2] ★

- LIMIT
- Nested queries / Uncorrelated subqueries ★
- Conditions IN / NOT IN / EXISTS / ALL / ANY
- INNER / OUTER / LEFT JOIN
- NULLs
- WITH statement

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

LIMITING results

RETURN only the TOP 3

```
SELECT *
FROM student
ORDER BY semester DESC;

SELECT *
FROM student
ORDER BY semester DESC
FETCH FIRST 3 ROWS ONLY;
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Standard SQL 2008
Supported by IBM DB2, Sybase SQL Anywhere, PostgreSQL,...

LIMITING results – OFFSET

RETURN only the TOP 3 after the first 2

```
SELECT *
FROM student
ORDER BY semester DESC
OFFSET 2 ROWS
FETCH FIRST 3 ROWS ONLY;
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Standard SQL 2008
Supported by IBM DB2, Sybase SQL Anywhere, PostgreSQL,...

LIMITING results – NON STANDARD

RETURN only the TOP 3 after the first 2

```
SELECT *
FROM student
ORDER BY semester DESC
OFFSET 2
LIMIT 3;
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Non standard syntax!

Supported by MySQL, Sybase SQL Anywhere, PostgreSQL,...

Nested queries – Subqueries

Complex comparisons that are hard to write with a join

- Subqueries are necessary for comparisons to sets of values
  - **Standard comparisons** in combination with quantifiers: ALL or ANY
  - **Special keywords** to access sets: IN and EXISTS

```
SELECT name
FROM professor
WHERE id IN (SELECT taughtby
             FROM course);
```

Names of the professors that teach at least one course

Uncorrelated subquery

Uncorrelated subqueries - IN

Determine the id of the professors with offices in the same building as professors from the department of Philosophy (PHI)

```
SELECT id
FROM professor
WHERE building IN (SELECT building
                   FROM professor
                   WHERE dept = 'PHI');
```

building
2
2
3

professor

id	name	rank	dept	building
2125	Socrates	C4	PHI	2
2126	Russel	C4	SCI	7
2128	Russel	C2	PHI	2
2133	Popper	C3	CS	9
2134	Augustinus	C3	THE	2
2137	Kant	C4	PHI	3

id
2125
2128
2134
2137

Uncorrelated subqueries - ANY

- **ANY** compares a value with a set of values
- The condition is true for **at least** one element of the set

```
SELECT name
FROM professor
WHERE rank > ANY (SELECT rank
                  FROM professor);
```

rank
C4
C4
C2
C3
C3
C4

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

name
Socrates
Russel
Popper
Augustinus
Kant

Uncorrelated subqueries - ALL

- **ALL** compares a value with a set of values
- The condition is true for **all** the elements of the set

```
SELECT name
FROM professor
WHERE rank >= ALL (SELECT rank
                   FROM professor);
```

rank
C4
C4
C2
C3
C3
C4

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

name
Socrates
Russel
Kant

Uncorrelated subqueries - ALL

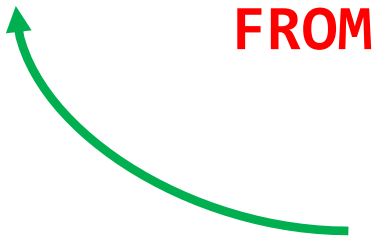
- **ALL** compares a value with a set of values
- The condition is true for **all** the elements of the set

rank
C4
C4
C2
C3
C3
C4

```
SELECT name
FROM professor
WHERE rank > ALL (SELECT rank
                  FROM professor);
```

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4



name

Uncorrelated subqueries – NOT IN

- **NOT IN reverses the IN check**
- Equivalent to form of INTESECTION operation

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

```
SELECT id
FROM professor
WHERE id NOT IN (SELECT taughtby
                 FROM course);
```

taughtby
2137
2125
2126

id
2128
2133
2134

Correlated Subqueries - Exists

- **The subqueries refers to values of the parent query**

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

Names of the professors that teach at least one course

```
SELECT name
FROM professor p
WHERE EXISTS (SELECT *
              FROM course c
              WHERE c.taughtby = p.id);
```

courseid	title	ects	taughtby
5041	Ethics	4	2125

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

Correlated Subqueries - Exists

- **The subqueries refers to values of the parent query**

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

Names of the professors that teach at least one course

```
SELECT name
FROM professor p
WHERE EXISTS (SELECT *
              FROM course c
              WHERE c.taughtby = p.id);
```

courseid	title	ects	taughtby
5043	Theory of Cognition	3	2126

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

Correlated Subqueries - Exists

- **The subqueries refers to values of the parent query**

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

Names of the professors that teach at least one course

```
SELECT name
FROM professor p
WHERE EXISTS (SELECT *
              FROM course c
              WHERE c.taughtby = p.id);
```

courseid	title	ects	taughtby

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

Correlated Subqueries - Exists

- **The subqueries refers to values of the parent query**

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

Names of the professors that teach at least one course

```
SELECT name
FROM professor p
WHERE EXISTS (SELECT *
              FROM course c
              WHERE c.taughtby = p.id);
```

courseid	title	ects	taughtby
5001	Basics	3	2137

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

name
Socrates
Russel
Kant

Which are valid queries?

A)

```
SELECT name
FROM professor p
WHERE EXISTS (SELECT 42
              FROM course c
              WHERE c.taughtby = p.id);
```

B)

```
SELECT name
FROM professor p
WHERE EXISTS (SELECT 42
              FROM course c );
```

C)

```
SELECT name
FROM professor p
WHERE ANY (SELECT 42
           FROM course c );
```

Correlated subqueries explained

- What is computed here?

??
42

??
42

??

??
42

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

```
SELECT name
FROM professor p
WHERE EXISTS (SELECT 42
              FROM course c
              WHERE c.taughtby = p.id);
```

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126

name
Socrates
Russel
Kant

IN vs. EXISTS

```
SELECT name
FROM professor
WHERE id IN (SELECT taughtby
             FROM course);
```

Is the “value” in the “right set”

```
SELECT name
FROM professor p
WHERE EXISTS (SELECT *
              FROM course c
              WHERE c.taughtby = p.id);
```

Is the “right set” nonempty?

JOIN Variants

Standard formulation

```
SELECT *
FROM R1, R2
WHERE R1.A = R2.B;
```

Alternative formulation (**INNER JOIN**)

```
SELECT *
FROM R1 JOIN R2
  ON R1.A = R2.B;
```

NATURAL JOIN

```
SELECT *
FROM R1 NATURAL JOIN R2;
```

Other variants:
LEFT, **RIGHT**, or **FULL OUTER JOIN**

INNER Join

List all the students that have been graded along with their grades

```
SELECT s.studid, name, semester, courseid, empid, grade
FROM student s INNER JOIN grades g
  ON s.studid = g.studid;
```

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

grades

studid	courseid	empid	grade
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2

studid	name	semester	courseid	empid	grade
25403	Jonas	12	5041	2125	2
27550	Schopenhauer	6	4630	2137	2

Remember the attribute names

professor(id, name, rank)

course(courseid, title, ects, **id** → professor)

```
SELECT id, name, title
FROM professor, course
WHERE professor.id = course.id;
```

Incorrect statement

The result of the join has two columns with name *id* (ambiguous reference)

```
SELECT professor.id, name, title
FROM professor, course
WHERE professor.id = course.id;
```

Correct statement

LEFT OUTER Join

List all the students that have been graded along with their grades

```
SELECT s.studid, name, semester, courseid, empid, grade
FROM student s LEFT OUTER JOIN grades g
  ON s.studid = g.studid;
```

grades

studid	courseid	empid	grade
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2

student

studid	name	semester	courseid	empid	grade
24002	Xenokrates	18	⊥	⊥	⊥
25403	Jonas	12	5041	2125	2
26120	Fichte	10	⊥	⊥	⊥
26830	Aristoxenos	8	⊥	⊥	⊥
27550	Schopenhauer	6	4630	2137	2

Outer join variants:
LEFT, **RIGHT**,
or **FULL OUTER JOIN**

LEFT OUTER Join (II)

List all the students that have been graded along with their grades

```
SELECT s.studid, name, semester, courseid, empid, grade
FROM student s LEFT OUTER JOIN grades g
  ON s.studid = g.studid;
```

grades

studid	courseid	empid	grade
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2
27550	5041	2125	2
28990	4630	2137	2

output

studid	name	semester	courseid	empid	grade
24002	Xenokrates	18	⊥	⊥	⊥
25403	Jonas	12	5041	2125	2
26120	Fichte	10	⊥	⊥	⊥
26830	Aristoxenos	8	⊥	⊥	⊥
27550	Schopenhauer	6	4630	2137	2

Joins & NULLs

What is one (not only) important difference between an outer join and a cartesian product?
Cartesian products does not produce NULL attributes

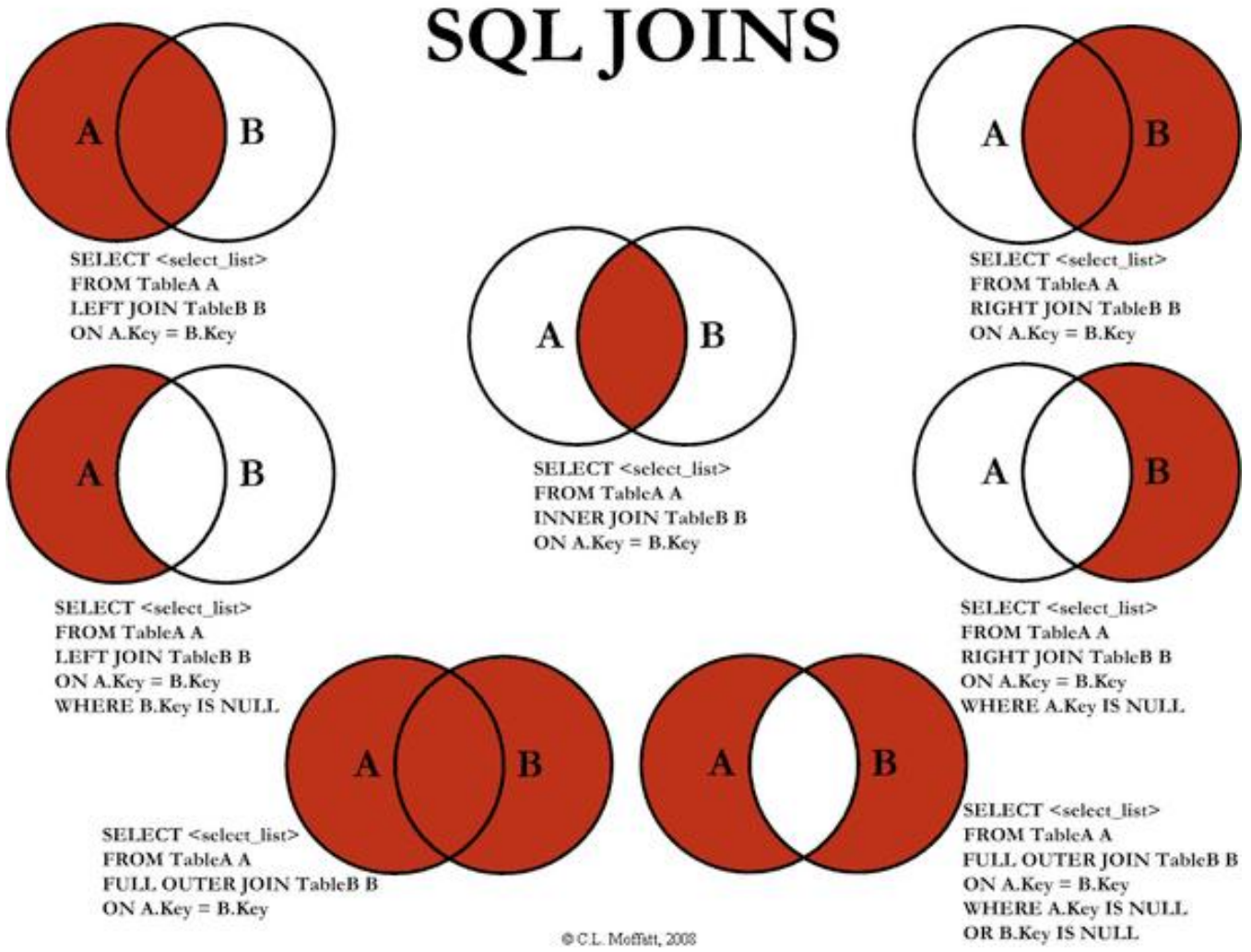
Types of Joins

The default joins only returns tuples that satisfy the condition.

There are other types of joins that preserve rows that do not match the condition (outer joins)!

OUTER JOINS

With outer joins, non matching values are replaced with NULLS



Remember: they are not set operations, this is just a visualization!

What is the evaluation result value of `grade > 1 AND grade < 12` when grade is NULL? unknown, i.e., NULL

- Arithmetic expressions: null values are "propagated"
  - NULL + 1 → NULL
  - NULL * 0 → NULL
- Algebraic Comparison operators (<,>, <>, =)
  - If at least one argument is NULL, then the result is **unknown**
  - grade < 12 → unknown whenever grade is NULL
  - **SQL has a three-valued logic:** true, false, and unknown

Evaluation of logical expressions

NOT	
true	false
unknown	unknown
false	true

AND	true	unknown	false
true	true	unknown	false
unknown	unknown	unknown	false
false	false	false	false

OR	true	unknown	false
true	true	true	true
unknown	true	unknown	unknown
false	true	unknown	false

Which of the following queries select all rows for which the semester is a null value?

A.
```
SELECT *
FROM student
WHERE semester IS NULL;
```

B.
```
SELECT *
FROM student
WHERE semester = NULL;
```

(A) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grade g
  ON s.studid = g.studid;
```

student

studid	name	semester
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

grade

studid	courseid	empid	grade
25403	5041	2125	2
27550	4630	2137	2
27550	5041	2125	2
28990	4630	2137	2

(A) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grade g
  ON s.studid = g.studid;
```

student

studid	name	semest
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

studid	courseid	grade
25403	5041	2
26120	NULL	NULL
26830	NULL	NULL
27550	4630	2
27550	5041	2
NULL	4630	2

e

udid	courseid	empid	grade
403	5041	2125	2
550	4630	2137	2
550	5041	2125	2
990	4630	2137	2

(B) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grades g
  ON s.studid = g.studid AND semester >8;
```

student

studid	name	semester
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

grade

studid	courseid	empid	grade
25403	5041	2125	2
27550	4630	2137	2
27550	5041	2125	2
28990	4630	2137	2

(B) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grades g
  ON s.studid = g.studid AND semester >8;
```

student

studid	name	semest
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

studid	courseid	grade
25403	5041	2
NULL	4630	2
NULL	5041	2
NULL	4630	2
26120	*NULL*	*NULL*
27550	*NULL*	*NULL*
26830	*NULL*	*NULL*

e

udid	courseid	empid	grade
403	5041	2125	2
550	4630	2137	2
550	5041	2125	2
990	4630	2137	2

(C) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grades g
  ON s.studid = g.studid
WHERE semester >8;
```

student

studid	name	semester
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

grade

studid	courseid	empid	grade
25403	5041	2125	2
27550	4630	2137	2
27550	5041	2125	2
28990	4630	2137	2

(C) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grades g
  ON s.studid = g.studid
WHERE semester >8;
```

student

studid	name	semest
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

studid	courseid	grade
25403	5041	2
26120	*NULL*	*NULL*

es

udid	courseid	empid	grade
403	5041	2125	2
550	4630	2137	2
27550	5041	2125	2
28990	4630	2137	2

(D) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grades g
  ON s.studid = g.studid
WHERE semester is NULL;
```

student

studid	name	semester
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

grade

studid	courseid	empid	grade
25403	5041	2125	2
27550	4630	2137	2
27550	5041	2125	2
28990	4630	2137	2

(D) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grade g
  ON s.studid = g.studid
WHERE semester is NULL;
```

student

studid	name	semest
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

studid	courseid	grade
NULL	4630	2

e

udid	courseid	empid	grade
403	5041	2125	2
27550	4630	2137	2
27550	5041	2125	2
28990	4630	2137	2

(E) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grade g
  ON s.studid = g.studid AND semester is NULL;
```

student

studid	name	semester
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

grade

studid	courseid	empid	grade
25403	5041	2125	2
27550	4630	2137	2
27550	5041	2125	2
28990	4630	2137	2

(E) What is the output

```
SELECT s.studid, courseid, grade
FROM student s FULL OUTER JOIN grade g
  ON s.studid = g.studid AND semester is NULL;
```

student

studid	name	semest
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6

studid	courseid	grade
NULL	5041	2
NULL	4630	2
NULL	5041	2
NULL	4630	2
25403	*NULL*	*NULL*
26120	*NULL*	*NULL*
27550	*NULL*	*NULL*
26830	*NULL*	*NULL*

e

udid	courseid	empid	grade
403	5041	2125	2
550	4630	2137	2
550	5041	2125	2
990	4630	2137	2

Complex Nesting – Always renaming in FROM

```
SELECT costs.empid, costs.cost
FROM (
 SELECT empid, months*salary AS cost
  FROM worker
  WHERE months >2 ) AS costs
WHERE cost >= ALL (
      SELECT months*salary AS cost
        FROM worker
        WHERE months >2
);
```

worker

empid	name	months	salary	cost
5001	Bob	4	20000	80000
5041	Anne	4	21000	84000
5043	Alex	3	35000	105000
~~5048~~	~~Jane~~	~~2~~	~~10000~~	
4052	Lars	4	15000	60000
5052	Mette	3	40000	120000
~~5210~~	~~Rikke~~	~~2~~	~~10000~~	

Result

empid	cost
5052	120000

Temporary data: WITH statement

To simplify complex queries!

Called common table expression (CTE)

```
WITH costs AS (
SELECT empid, months*salary AS cost
  FROM worker
  WHERE months > 2
)
SELECT empid, cost
FROM costs
WHERE cost >= ALL (
      SELECT cost
        FROM costs
);
```

ONLY 1 STATEMENT!

worker

empid	name	months	salary
5001	Bob	4	20000
5041	Anne	4	21000
5043	Alex	3	35000
~~5048~~	~~Jane~~	~~2~~	~~10000~~
4052	Lars	4	15000
5052	Mette	3	40000
~~5210~~	~~Rikke~~	~~2~~	~~10000~~

costs

empid	cost
5001	80000
5041	84000
5043	105000
4052	60000
5052	120000

not stored anywhere!

Result

empid	cost
5052	120000

Outline this lecture

Data Manipulation Language [part 2] ★

- Insert (+ select)
- Update (+where)
- Referential Integrity

Data Definition Language [part 2]

- CREATE (as query / with data) ★
- Sequence numbers
- Cascade / set NULL / set default ★

Data Query Language [part 2] ★

- LIMIT
- Nested queries / Uncorrelated subqueries
- Conditions IN / NOT IN / EXISTS / ALL / ANY
- INNER / OUTER / LEFT JOIN
- NULLs
- WITH statement

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

Best way to learn is to try!

- Find the exercises on Moodle
- Try to solve them
- Install Postgres (some help is on moodle), try to recreate examples and exercises
- You can use online tools also https://extendsclass.com/postgresql-online.html